BAB 5
PEWARISAN

Capaian:

Setelah menempuh bab ini, diharapkan mahasiswa mampu menguasai kompetensi berikut:

1. Mampu menjelaskan konsep pewarisan pada pemrograman berbasis objek
2. Mampu mengimplementasikan konsep pewarisan dalam pemrograman python

Tujuan Pembelajaran:

Tujuan dari pembelajaran ini adalah mahasiswa dapat:

1. Membuat kelas induk (*superclass*) dan kelas anak (*subclass*) dari entitas yang ada di dunia nyata
2. Mengimplementasikan overriding
3. Membuat kelas abstrak

Praktikum 5.1: Membuat Sebuah Pewarisan

Sebelum memulai praktikum, pastikan seluruh peralatan dalam kondisi baik dan lingkungan kerja aman serta nyaman. Selalu perhatikan prinsip K3 dengan menjaga keselamatan, kesehatan, dan keamanan data selama kegiatan praktikum berlangsung. Jika semua sudah terpenuhi, perhatikan langkah-langkah praktikum berikut ini:

1) Buat file python pada project Anda dengan nama **praktikum_5.1.py**!
2) Buat sebuah kelas dengan nama Hewan. Selanjutnya buat kelas Invertebrata sebagai subclass dari Hewan. Buat pula kelas Mollusca sebagai subclass dari Invertebrata seperti berikut ini:

```
class Hewan:
    pass

class Invertebrata(Hewan):
    pass

class Molusca(Invertebrata):
    pass
```

3) Buat pula sebuah kelas Vertebrata sebagai subclass dari Hewan. Selanjutnya, buat kelas Mamalia sebagai subclass dari Vertebrata dan kelas Kucing sebagai subclass dari Mamalia seperti berikut ini:

```
class Vertebrata(Hewan):
    pass

class Mamalia(Vertebrata):
    pass
```

```
class Kucing (Mamalia):
    pass
```

4) Periksa apakah sebuah kelas adalah subclass dari kelas tertentu menggunakan fungsi **issubclass()** seperti berikut ini:

```
print('Apakah','Kucing','Subclass dari',
'Mamalia','?', issubclass(Kucing, Mamalia))
print('Apakah','Kucing','Subclass dari',
'Vertebrata','?', issubclass(Kucing, Vertebrata))
print('Apakah','Kucing','Subclass dari',
'Invertebrata','?', issubclass(Kucing, Invertebrata))
```

Jika code tersebut dijalankan, maka akan tampil hasil sebagai berikut:

```
<n3 "/Users/nurfarida/Documents/Kerja 2025/Pengajaran/I
Apakah Kucing Subclass dari Mamalia ? True
Apakah Kucing Subclass dari Vertebrata ? True
Apakah Kucing Subclass dari Invertebrata ? False
```

5) Buat objek dari kelas Kucing, Mamalia, Vertebrata, dan Invertebrata:

```
kucing = Kucing()
mamalia = Mamalia()
vertebrata = Vertebrata()
invertebrata = Invertebrata()
```

6) Periksa apakah sebuah objek terbentuk dari kelas tertentu menggunakan **isinstance()**:

```
print('Apakah', 'kucing', 'instance dari ', 'Kucing',
'?', isinstance(kucing, Kucing))
print('Apakah', 'kucing', 'instance dari ', 'Mamalia',
'?', isinstance(kucing, Mamalia))
print('Apakah', 'kucing', 'instance dari ',
'Vertebrata', '?', isinstance(kucing, Vertebrata))
```

```
print('Apakah', 'kucing', 'instance dari ',
'Invertebrata', '?', isinstance(kucing, Invertebrata))
```

Jika code tersebut dijalankan, maka akan tampil hasil sebagai berikut:

```
Apakah kucing instance dari  Kucing ? True
Apakah kucing instance dari  Mamalia ? True
Apakah kucing instance dari  Vertebrata ? True
Apakah kucing instance dari  Invertebrata ? False
```

Jadi, apa perbedaan antara fungsi **issubclass()** dan **isinstance()**?

Praktikum 5.2: Mengenal Fungsi super()

Praktikum selanjutnya adalah mengenal bagaimana cara menggunakan fungsi super() dalam pewarisan. Perhatikan langkah-langkah praktikum berikut ini:

1) Buat file python pada project Anda dengan nama **praktikum_5.2.py**!
2) Buat kelas Mamalia yang terdiri dari konstruktor __init__() dan method __str__(). Kemudian buat kelas Kucing (yang mewarisi kelas Mamalia). Panggil konstruktor Mamalia dalam konstruktor Kucing seperti berikut ini:

```
class Mamalia:
    def __init__(self, nama):
        self.nama = nama
    def __str__(self):
        return "Nama : "+ self.nama + "."

class Kucing (Mamalia):
    def __init__ (self, nama):
        Mamalia.__init__(self, nama)
```

3) Buat objek kucing dengan parameter masukan “Kiwi” dengan nama obj1. Selanjutnya beri perintah print() untuk obj1 seperti berikut ini:

```
obj1 = Kucing("Kiwi")
print (obj1)
```

Jika code tersebut dijalankan, maka akan tampil hasil sebagai berikut:

```
nurfarida@Ellyas-MacBook-Pro Bab 5 % /usr/local/bin/
u PBO/Bab 5/praktikum_5_2.py"
Nama : Kiwi.
```

Dari sini apa yang dapat Anda simpulkan?

4) Ubah pemanggilan konstruktor mamalia pada konstruktor Kucing menggunakan syntax **super()** seperti berikut ini:

```
class Mamalia:
    def __init__(self, nama):
        self.nama = nama
    def __str__(self):
        return "Nama : "+ self.nama + "."

class Kucing (Mamalia):
    def __init__ (self, nama):
        super().__init__(nama)
```

5) Buat objek kucing dengan parameter masukan “Berry” dengan nama obj2. Selanjutnya beri perintah print() untuk obj2 seperti berikut ini:

```
obj2 = Kucing("Berry")
print(obj2)
```

Jika code tersebut dijalankan, maka akan tampil hasil sebagai berikut:

```
nurfarida@Ellyas-MacBook-Pro Bab 5 % /usr/local/bin/
u PBO/Bab 5/praktikum_5_2.py"
Nama : Berry.
```

Praktikum 5.3: Membuat Class Variable dan Instance Variable

Praktikum selanjutnya adalah mengenal bagaimana cara membuat *class variable* dan *instance variable*. Perhatikan langkah-langkah praktikum berikut ini:

1) Buat file python pada project Anda dengan nama **praktikum_5.3.py**!
2) Buat kelas Mamalia yang memiliki *class variable* paruparu dan konstruktor. Buat pula kelas Kucing (yang mewarisi kelas Mamalia) dan memiliki *class variable* kaki dan konstruktor seperti berikut ini:

```
class Mamalia:
    paruparu = True
    def __init__(self,var):
        self.nama = val

class Kucing(Mamalia):
    kaki = 4
    def __init__(self,val):
        self.nama =val
```

3) Buat sebuah objek obj1 dan obj2 dari kelas kucing dan tampilkan *class variable* tersebut seperti berikut ini:

```
obj1=Kucing('Kiwi')
print(obj1.nama)
print(obj1.kaki)
print(obj1.paruparu)

obj2=Kucing('Berry')
print(obj2.nama)
print(obj2.kaki)
```

```
print(obj2.paruparu)
```

Jika code tersebut dijalankan, maka akan tampil hasil sebagai berikut:

```
<ran/buku PBO/Bab 5/praktikum_5_3.py"
Kiwi
4
True
Berry
4
True
```

Perhatikan bahwa *class variable* tidak perlu didefinisikan ketika membuat objek dan selalu bernilai sama.

4) Buat kembali kelas Mamalia yang memiliki konstruktor yang didalamnya terdapat *instance variable* paruparu. Buat pula kelas Kucing (yang mewarisi kelas Mamalia) yang memiliki *instance variable* kaki seperti berikut ini:

```
class Mamalia:
    def __init__(self):
        self.paruparu = True

class Kucing(Mamalia):
    def __init__(self,val1, val2, val3):
        super().__init__()
        self.nama=val1
        self.kaki=val2
        self.warna=val3
```

5) Buat sebuah objek obj1 dan obj2 dari kelas kucing dan tampilkan *instance variable* tersebut seperti berikut ini:

```
obj1=Kucing('Kiwi',4,'Golden')
print(obj1.nama)
```

```
print(obj1.kaki)
print(obj1.paruparu)
obj2=Kucing('Berry',4, 'Abu')
print(obj2.nama)
print(obj2.kaki)
print(obj2.paruparu)
```

Jika code tersebut dijalankan, maka akan tampil hasil sebagai berikut:

```
nurfarida@Ellyas-MacBook-Pro Bab 5 %
uments/Kerja 2025/Pengajaran/buku PBO
Kiwi
4
True
Berry
4
True
```

Variabel ini disebut dengan variable instance (*instance variable*)

6) Buat kelas Mamalia yang memiliki *class variable, instance variable,* dan method. Buat pula kelas Kucing yang memiliki *class variable*, *instance variable,* dan method seperti berikut ini:

```
class Mamalia():
    paruparu = True
    def __init__(self):
        self.gigi = True

    def bergerak(self):
        return 'Berjalan/Berenang'

class Kucing(Mamalia):
    ekor = True
    def __init__(self, val1,val2, val3):
        self.nama = val1
        self.kaki = val2
        self.suara=val3

    def bersuara(self):
```

```
        return self.suara
```

7) Buat sebuah objek dari kelas kucing. Panggil dan tampilkan semua properti dari kelas kucing. Panggil dan tampilkan juga semua properti dari kelas mamalia yang diturunkan ke kelas kucing seperti berikut ini:

```
obj = Kucing('Kiwi',4,'Meow')
print('Suara : ',obj.bersuara())
print('Kaki  : ',obj.kaki)
print('Ekor  : ',obj.ekor)
print('Bergerak    : ',obj.bergerak())
print('Paru-paru   : ',obj.paruparu)
print('Gigi        : ',obj.gigi)
```

Jika code tersebut dijalankan, maka akan tampil hasil sebagai berikut:

```
<Kerja 2025/Pengajaran/buku PBO/Bab 5/praktikum_5_3.py"
Suara :  Meow
Kaki  :  4
Ekor  :  True
Bergerak    :  Berjalan/Berenang
Paru-paru   :  True
Traceback (most recent call last):
  File "/Users/nurfarida/Documents/Kerja 2025/Pengajaran/bu
ku PBO/Bab 5/praktikum_5_3.py", line 93, in <module>
    print('Gigi        : ',obj.gigi)
                         ^^^^^^^^
AttributeError: 'Kucing' object has no attribute 'gigi'
```

Terjadi error ketika mengakses properti gigi. Hal ini karena gigi merupakan instance variabel milik mamalia yang tidak pernah dibentuk ketika membentuk objek kucing.

8) Tambahkan code untuk memanggil konstruktor milik kelas induk yakni super() seperti berikut ini:

```
class Mamalia():
```

```
    paruparu = True
    def __init__(self):
        self.gigi = True

    def bergerak(self):
        return 'Berjalan/Berenang'

class Kucing(Mamalia):
    ekor = True
    def __init__(self, val1,val2, val3):
        super().__init__()      #memanggil konstruktor
superclass
        self.nama = val1
        self.kaki = val2
        self.suara=val3

    def bersuara(self):
        return self.suara
```

Jika code tersebut dijalankan, maka akan tampil hasil sebagai berikut:

```
nurfarida@Ellyas-MacBook-Pro Bab 5 % /
 "/Users/nurfarida/Documents/Kerja 202
/Bab 5/praktikum_5_3.py"
Suara :  Meow
Kaki  :  4
Ekor  :  True
Bergerak    :  Berjalan/Berenang
Paru-paru   :  True
Gigi        :  True
```

Praktikum 5.4: Membuat Method Overriding

Praktikum selanjutnya adalah mengenal bagaimana cara membuat method overriding. Perhatikan langkah-langkah praktikum berikut ini:

1) Buat file python pada project Anda dengan nama **praktikum_5.4.py**!
2) Buat kelas Kucing dan Sapi yang memiliki properti dengan nama yang sama. Buat objek dari kelas kucing dan sapi, kemudian panggil masing-masing properti seperti berikut ini:

```
class Kucing():
    warna = 'Kuning'
    def bersuara(self):
        return 'meow'

class Sapi():
    warna = 'Putih'
    def bersuara(self):
        return 'Moo'

Kiwi = Kucing()
print(Kiwi.warna)
print(Kiwi.bersuara())

api = Sapi()
print(api.warna)
print(api.bersuara())
```

3) Buat kembali kelas Mamalia, Kucing (yang mewarisi kelas Mamalia), dan British (yang mewarisi kelas Kucing).

Selanjutnya buat objek dari kelas British, kemudian panggil properti dari kelas supernya seperti berikut ini:

```
class Mamalia():
    ekor = 'Ada / Tidak ada'
    def bergerak(self):
        return 'Berjalan/ Berenang'

class Kucing(Mamalia):
    ekor ='Ada'
    def bergerak(Self):
        return 'Berjalan'

class British (Kucing):
    pass

Kiwi = British()
print(Kiwi.ekor)
print(Kiwi.bergerak())
```

Jika code tersebut dijalankan, maka akan tampil hasil sebagai berikut:

```
nurfarida@Ellyas-MacBook-Pro Bab 5 %
ran/buku PBO/Bab 5/praktikum_5_4.py"
Ada
Berjalan
```

Properti milik kelas mana yang terpanggil?

4) Buat kelas British dan Ragdoll yang memiliki beberapa properti yang sama. Buat pula kelas Campuran **yang mewarisi kelas British dan Ragdoll**. Selanjutnya buat objek dari kelas Campuran, kemudian panggil semua properti sebagai berikut:

```
class British:
    warna = 'Kuning'
    berat = True
    def sifat(self):
```

```
        return 'Lincah'

class Ragdoll:
    warna = 'Putih'
    tinggi = True
    def sifat(self):
        return 'Tenang'

class Campuran (British, Ragdoll):
    pass

Kiwi = Campuran()
print(Kiwi.warna)
print(Kiwi.berat)
print(Kiwi.tinggi)
print(Kiwi.sifat())
```

Jika code tersebut dijalankan, maka akan tampil hasil sebagai berikut:

```
nurfarida@Ellyas-MacBook-Pro Bab 5 %
ran/buku PBO/Bab 5/praktikum_5_4.py"
Kuning
True
True
Lincah
```

Properti milik kelas mana yang terpanggil? Mengapa demikian?

5) Ubah urutan pewarisan pada kelas Campuran menjadi **Ragdoll dan British** seperti berikut ini:

```
class British:
    warna = 'Kuning'
    berat = True
    def sifat(self):
        return 'Lincah'

class Ragdoll:
    warna = 'Putih'
    tinggi = True
    def sifat(self):
```

```
        return 'Tenang'

class Campuran (Ragdoll, British):
    pass

Kiwi = Campuran()
print(Kiwi.warna)
print(Kiwi.berat)
print(Kiwi.tinggi)
print(Kiwi.sifat())
```

Jika code tersebut dijalankan, maka akan tampil hasil sebagai berikut:

```
nurfarida@Ellyas-MacBook-Pro Bab 5 %
ran/buku PBO/Bab 5/praktikum_5_4.py"
Putih
True
True
Tenang
```

Properti milik kelas mana yang terpanggil? Mengapa demikian?

Praktikum 5.5: Membuat Kelas Abstrak

Praktikum selanjutnya adalah mengenal bagaimana cara membuat kelas abstrak. Perhatikan langkah-langkah praktikum berikut ini:

1) Buat file python pada project Anda dengan nama **praktikum_5.5.py**!
2) Import **ABC** dan **abstractmethod** dari modul **abc**. Kemudian buat sebuah kelas abstrak Hewan seperti berikut ini:

```
from abc import ABC, abstractmethod

class Hewan(ABC):
    @abstractmethod
    def tulangbelakang(self):
        pass

    @abstractmethod
    def organnafas(self):
        pass

    @abstractmethod
    def kaki(self):
        pass
```

3) Buat sebuah objek dari kelas abstrak tersebut seperti berikut ini:

```
hewan = Hewan()
```

Jika code tersebut dijalankan, maka akan tampil hasil sebagai berikut:

```
nurfarida@Ellyas-MacBook-Pro Bab 5 % /usr/local/b
in/python3 "/Users/nurfarida/Documents/Kerja 2025
/Pengaja
ran/buku PBO/Bab 5/praktikum_5_5.py"
Traceback (most recent call last):
  File "/Users/nurfarida/Documents/Kerja 2025/Pen
gajaran/buku PBO/Bab 5/praktikum_5_5.py", line 16
, in <module>
    hewan = Hewan()
            ^^^^^^^
TypeError: Can't instantiate abstract class Hewan
 without an implementation for abstract methods '
kaki', 'organnafas', 'tulangbelakang'_
```

Mengapa terjadi error?

4) Buat sebuah kelas Kucing, Ikan, dan Udang yang mewarisi kelas abstrak Hewan seperti berikut ini:

```
class Kucing(Hewan):
    def tulangbelakang(self):
        return True
    def organnafas(self):
        return 'Paru-paru'
    def kaki(self):
        return 4

class Ikan(Hewan):
    def tulangbelakang(self):
        return True
    def organnafas(self):
        return 'Insang'
    def kaki(self):
        return 0

class Udang(Hewan):
    def tulangbelakang(self):
        return False
    def organnafas(self):
        return 'Insang'
    def kaki(self):
        return 10
```

5) Buat sebuah objek dari kelas Kucing, Ikan, dan Udang seperti berikut ini:

```
kucing = Kucing()
print(kucing.tulangbelakang())
print(kucing.organnafas())
print(kucing.kaki())

ikan = Ikan()
print(ikan.tulangbelakang())
print(ikan.organnafas())
print(ikan.kaki())

udang = Udang()
print(udang.tulangbelakang())
print(udang.organnafas())
print(udang.kaki())
```

Jika code tersebut dijalankan, maka akan tampil hasil sebagai berikut:

```
nurfarida@Ellyas-MacBook-Pro Bab 5 % /usr
5/Pengajaran/buku PBO/Bab 5/praktikum_5_5
True
Paru-paru
4
True
Insang
0
False
Insang
10
```

6) Buat kembali sebuah kelas Burung yang mewarisi kelas abstrak Hewan seperti berikut ini:

```
class Burung(Hewan):
    def tulangbelakang(self):
        return True
    def organnafas(self):
        return 'paru-paru dan pundi-pundi udara'
```

7) Buat sebuah objek dari kelas burung seperti berikut ini:

```
burung = Burung()
print(burung.tulangbelakang())
print(burung.organnafas())
```

Jika code tersebut dijalankan, maka akan tampil hasil sebagai berikut:

```
nurfarida@Ellyas-MacBook-Pro Bab 5 % /usr/local/bin/py
thon3 "/Users/nurfarida/Documents/Kerja 202
5/Pengajaran/buku PBO/Bab 5/praktikum_5_5.py"
Traceback (most recent call last):
  File "/Users/nurfarida/Documents/Kerja 2025/Pengajar
an/buku PBO/Bab 5/praktikum_5_5.py", line 63, in <modu
le>
    burung = Burung()
             ^^^^^^^^
TypeError: Can't instantiate abstract class Burung wit
hout an implementation for abstract method 'kaki'
```

Mengapa terjadi error?

Latihan

Jawablah pertanyaan di bawah ini sebagai pengayaan Bab 5!

1. Mengapa konsep pewarisan dapat membantu mengurangi penulisan kode yang berulang? Jelaskan.
2. Apa fungsi `super()` pada Python? Jelaskan kapan fungsi tersebut digunakan.
3. Dalam suatu program, subclass memiliki method yang sama dengan superclass tetapi isi method berbeda. Analisis mengapa overriding diperlukan pada kondisi tersebut!
4. Menurut Anda, kapan penggunaan inheritance lebih efektif dibandingkan membuat class terpisah? Jelaskan dengan contoh kasus.
5. Sebuah programmer sedang merancang sistem informasi perusahaan transportasi berbasis OOP. Pengembang tersebut merancang sebuah kelas abstrak **Kendaraan** yang memiliki variabel kelas **jumlah_kendaraan** untuk menghitung total objek, serta variabel instance **nama** dan **kecepatan**. Kelas ini memiliki method abstrak **bergerak()** yang harus diimplementasikan oleh kelas turunan **Mobil** dan **Motor** sesuai karakteristiknya. Setiap pembuatan objek kendaraan akan menambah jumlah kendaraan.
    a. Buatlah rancangan diagram kelas sesuai kasus diatas!
    b. Implementasikan rancangan Anda dalam bahasa pemrograman python!
    c. Buat banyak objek dari masing-masing kelas turunan!